FUNDAMENTAL OF AI AND ML

CSA2001 ----- Course Project Report

Smart Expense Tracker

A CLI-Based AI Application For Personal Finance Management

Submitted by

Student Name	Abhinandan Kushwaha
Registration no.	25BAI11121
Course	Fundamental of AI and ML – CSA2001
Institution	VIT Bhopal University
Branch	B.Tech CSE (AI/ML)
Academic Year	2025-26(Semester II)
Submission Date	March 2026

Table of Contents

1. Title
2. Introduction
3. Motivation of the Project
4. Problem Statement
5. Objectives of the Project
6. Existing Methods
7. Pros and Cons of Stated Methods
8. Hardware and Software Requirements
9. Methodology and Goal
10. Algorithm Development
11. Output
12. Key Implementations Outline
13. Significant Project Outcomes
14. Testing and Refinement
15. Real-World Applicability
16. Contribution / Findings
17. Limitations / Constraints
18. Conclusions
19. Future Enhancements
20. References

1. Title

Smart Expense Tracker — A CLI-Based AI Application for Intelligent Personal Expense Classification and Spending Prediction

2. Introduction

The Smart Expense Tracker is an AI-assisted application for personal financial management, designed to be fully terminal-based. This application is developed as part of the CSA2001 course on Fundamentals of AI and ML. This application is coded using only Python, where users can add their daily expenses, automatically categorize their expenses using keyword-based AI logic, view the formatted details of all expenses, and predict their future expenses using linear regression.

This application is designed using the Python programming language, where the application is divided into different modules with clear separation of concerns. This application is designed to store data locally using a JSON file, where the application can store user expenses without relying on any centralized server. This application is an example of applying the concepts learned from the AI/ML course, where the application is designed using rule-based classification systems, supervised learning, and linear regression.

3. Motivation Of the Project

The motivation behind this project is based on a very common problem faced by people in their day-to-day life. The problem is the absence of simple, easy-to-use tools for personal expense tracking, which can also give intelligent results. All the expense

tracking tools that are currently available require a smartphone, internet access, or a complex account configuration.

Moreover, none of the tools currently available can give any intelligent results. They can only store data. As a student learning AI and ML, the goal of this project was to make something that goes one step beyond just storing data and actually uses AI techniques like categorizing expenses without asking the user to manually tag each expense, and even predict future expenses based on patterns.

Personal finance management is a problem that is commonly faced by people. This problem is commonly observed during the day-to-day life of a student, where it is very tedious for a person to manually keep track of their food, transport, and entertainment expenses.

4. Problem Statement

The process is prone to errors, time-consuming, and does not offer intelligent analysis. The current options are either over-engineered, requiring the user to have accounts, apps, or internet access, or they are too basic, using only a spreadsheet with no AI functionality. There is a need for an expense tracking tool that is lightweight, does not require an internet connection, is powered by AI, and can:

- Automatically classify expenses without requiring the user to manually tag them
- Persistently store expense information without relying on user sessions
- Make predictions about future expenses using data-driven predictions
- Be fully functional within a terminal without requiring any external infrastructure

---

The challenge is to implement AI classification, machine learning predictions, while keeping it simple enough for a non-technical user to still feel comfortable using it with a numbered menu.

5. Objectives of the Project

1. Develop a fully functional, menu-driven CLI application using Python
2. Utilize keyword-based AI classification for auto-categorization of expense descriptions
3. Implement linear regression using NumPy for predicting future expenditure patterns
4. Ensure all expense-related data is saved persistently using JSON files
5. Ensure robust validation of inputs for all user interactions
6. Ensure clean, readable, well-documented code, well-organized into logical modules
7. Ensure the practical applicability of AI/Machine Learning concepts is demonstrated for real-life personal finance management.

6. Existing Methods

6.1 Manual Spreadsheet Tracking

The most common method for personal expense tracking is to use a spreadsheet, such as Microsoft Excel or Google Sheets. This method is flexible, although it does require manual intervention for all expense tracking. This method does not utilize any form of AI.

6.2 Mobile Applications (Walnut, Money Manager, YNAB)

Mobile applications for expense tracking, such as Walnut, Money Manager, and YNAB, utilize AI to parse bank SMS messages, make expense category suggestions, and display expense tracking graphs. However, mobile applications require internet access, registration, and often pose a threat to user data, as the application is hosted on a server and not on the mobile device.

6.3 Database-Backed Desktop Applications

Applications like GnuCash are desktop applications for expense tracking and utilize a double-entry accounting system. This method is powerful, although it is not feasible for a student to utilize for expense tracking due to its complexity.

6.4 Pure CLI Tools (ledger-cli)

Tools like ledger-cli are text file-based accounting applications, and they are very flexible. However, they require a specific syntax to be used and do not utilize AI for classification and prediction.

7. Pros and Cons of Stated Methods

Method	Pros	Cons
Spreadsheet	Flexible, no installation	No AI, all manual
Mobile Apps	Automatic SMS parsing, dashboards	Requires internet, privacy risk
Desktop Tools	Full accounting features	Complex, heavy
ledger-cli	Lightweight, terminalnative	No AI, complex syntax

8. Hardware and Software Requirements

8.1 Hardware Requirements

- Any computer that will run Python 3.x (minimum RAM: 1 GB)
- No GPU or other special hardware required
- At least 50 MB free disk space

8.2 Software Requirements

- Python 3.8 or later versions
- NumPy library (install using pip install numpy)
- Terminal emulator (e.g., Command Prompt, PowerShell, bash, zsh)
- Git (optional: used to clone this repository)

No internet connection is required to run this program because it stores data locally.

9. Methodology and Goal

The project adheres to a module-based development pattern, where each of the processes, including user interface, AI classification, ML prediction, storage, and validation, is implemented as a Python module. The overall flow of the program is as follows:

1. The user executes the program by typing python main.py into the terminal

2. The existing expense data is loaded from the JSON storage by the CLI menu module (cli/menu.py)
3. The user is presented with a numbered list of menu options
4. For the addition of a new expense, validation of the amount, description, and determination of the category are carried out
5. The newly added expense is appended to the data list and saved immediately
6. For prediction, the predictor module uses NumPy's linear regression function to predict the future amount based on the recorded amounts
7. The program saves the data and displays a message before terminating

The aim of this program is to ensure a minimal, self-contained AI program that is actually useful for day-to-day life, as well as providing a tangible example of AI and ML concepts.

11. Algorithm Development

11.1 Keyword-Based Classification Algorithm

The classifier employs a dictionary-based lookup method, where the predefined mapping is from category names to keywords. The steps involved are:

1. Converts the user-provided description to lowercase
2. Iterates over each category in the keyword dictionary
3. Checks if any keyword for that category is present as a substring in the description
4. Returns the first matching category, or 'Other' if no keyword matches

This is an example of a deterministic classifier, which is an essential technique used in Artificial Intelligence. This technique is appropriate for the classification task because the vocabulary used in describing personal expenses is clearly bounded.

11.2 Linear Regression Prediction Algorithm

The predictor employs NumPy's polyfit function to fit a first-degree polynomial (a straight line) to the list of expense values recorded:

- x-axis: position of each expense (0, 1, 2, ...)
- y-axis: amount of each expense in rupees

The straight line is evaluated at position n (where n is the total number of existing expenses) to predict the amount of the next expense. This is an example of supervised learning, which is an essential technique used in Artificial Intelligence.

12. Output

12 . 1 Adding an expense

```
4. Help
5. Exit
Enter choice: 1
Enter amount (Rs): 425
Enter description: soda
[Success] Added Rs. 425.00 under 'Other'.

Press Enter to continue...[
```

12.2 Viewing Expense

```
5. Exit
Enter choice: 2
#    Amount (Rs)  Date         Category        Description
-----------------------------------------------------------
1      500000.00  2026-03-24    Other           ps5
2    5000000000000.00  2026-03-24    Other           property
3         425.00  2026-03-29    Other           soda

Press Enter to continue...[
```

12.3 Spendiong Prediction

```
1. Add Expense
2. View Expenses
3. Predict Spending
4. Help
5. Exit
Enter choice: 3
Predicted next expense: Rs. 1666666333900.0

Press Enter to continue...[
```

13. Key Implementation Outline of the System

- Modular Python code structure with proper separation between cli/, core/, storage/, utils/ directories
- Dictionary-based keyword classifier with 6 expense categories and 50+ keywords
- NumPy's polyfit function to implement linear regression and predict spending trends
- JSON-based data storage with immediate write to storage after each expense entry
- Input validation: numeric amount check, positive value check, and description check
- Proper error handling using try/except blocks at the function and menu levels
- Auto-creation of data/ directory if it doesn't already exist using (os.makedirs with exist_ok=True)
- Absolute paths in filehandler.py to ensure consistent operation regardless of directory from which script is run.

14. Significant Project Outcomes

- Successfully showcased AI keyword classification using a real-world financial scenario
- Developed a working ML prediction pipeline using linear regression on real user data
- Created a fully functioning CLI application that is error-resistant and saves data between sessions
- Successfully separated code into modules where every Python file has a single responsibility
- All features in the application (add, view, predict, help, exit) are functioning correctly without any GUI library used

15. Testing and Refinement

17.1 Input Validation Testing

The following cases have been manually tested:

- Non-numeric amount provided, say 'abc'. This will raise a ValueError, and the user will be asked to provide the amount again.
- Negative or zero amounts will be rejected by validate_amount() and will display an error message.
- An empty description will be rejected and will display the message 'Description cannot be empty'.
- An empty dataset will be provided to make a prediction, and the user will be asked to provide at least 2 records.

17.2 Storage Testing

Following cases have been tested:

- It has been ensured that the data will persist even after the restart of the application by loading data from expense.json.
- Case of a corrupt/missing JSON file: The application will start with a fresh list.

17.3 Classification Testing

Following cases have been tested:

- Descriptive phrases such as 'lunch at zomato', 'ola cab', 'recharge jio' will be classified correctly.
- Descriptive phrases such as 'miscellaneous payment' will also be classified correctly as 'Other'.

16. Project Applicability on Real-World Applications

- Student budgeting: it assists college students in managing their daily expenses without the need to install a mobile app on their smart devices
- Offline environments: it is fully functional without an internet connection, making it an ideal application in environments with low internet connectivity
- Small business expense logging: it can be used to assist small businesses in tracking their petty cash expenses
- Embedded systems/Raspberry Pi: its CLI nature makes it an ideal application to be used on embedded systems/Raspberry Pi devices
- Building blocks for larger systems: it provides an opportunity to expand its architecture to include a web interface, database system, and machine learning model updates

17. Contribution / Findings of the Project

The main technical contribution of this project is to show that it is possible to effectively use AI and ML techniques in a minimal, terminal-based scenario without the need for deep learning frameworks, GPU acceleration, or cloud services.

Findings from the development process:

- Rule-based keyword classifiers are very effective for bounded-vocabulary classification problems such as expense categorisation, having low computational overhead and good interpretability
- Linear regression, despite being a very simple model, is a useful spending trend predictor if at least 5-10 data points are available
- Modular code organisation greatly helped in reducing the time spent in debugging the code; separating the classifier and predictor made them independently testable
- JSON is enough as a persistence model for a CLI application of this type and intended user count, and database overhead is not necessary

18 Limitations/ Constraints of the System

- The keyword classifier is rule-based and cannot learn from new data; if a new category is to be added, the source code must be edited by hand
- The linear regression predictor assumes a linear trend in spending, which might not always be the case if the spending is irregular
- There is no multi-user capability; all data is stored in a shared JSON file
- The program has no graphical interface and might not be accessible to nontechnical users
- No date-based or category-based summaries are available in the view mode
- The predictor requires at least 2 data points and becomes more accurate as more data is available

19. Conclusions

The Smart Expense Tracker application successfully meets all the objectives outlined. The application serves to show that concepts of AI and ML, such as classification using a set of rules and linear regression, can be applied to a real-world application without undue complexity or infrastructure costs.

The application also serves to reinforce one of the key concepts learned throughout the AI/ML course: the appropriateness of a given algorithm depends on the problem constraints. For a classification problem with a constrained vocabulary, a keywordbased approach will always be superior to a more complex approach in terms of speed, simplicity, and ease of maintenance. For a small time-series data set, linear regression will always be a superior approach to deep learning techniques.

The application is fully functional, well tested, and modularly designed for use in a live environment. It represents a realistic application of concepts learned throughout the course to a problem that is immediately relevant to a student's everyday life.

20. Future Enhancements

- Replace keyword classifier with a trained Naive Bayes or TF-IDF based text classifier for improved accuracy on unseen descriptions
- Add category-wise monthly summary and bar chart visualisation using matplotlib
- Implement date-range filtering (e.g., view expenses for the current month only)
- Add multi-user support with separate JSON files per user profile
- Integrate a budget alert system that warns when spending in a category exceeds a set threshold
- Export expenses to CSV for use in spreadsheet applications
- Build a web frontend using Flask to provide a GUI version alongside the CLI

21. References

1. Harris, C.R. et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

2. Python Software Foundation. (2024). Python 3 Documentation. https://docs.python.org/3/

4. NumPy Documentation — numpy.polyfit. https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html

5. Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

6. Mitchell, T. (1997). Machine Learning. McGraw-Hill.

7. Python json module documentation.